

Peer-Review 1: UML

Didio Rocco – Boumchita Zakaria – Carbone Ferdinando – Del Vecchio Vincenzo

AM39

Valutazione del diagramma UML delle classi del gruppo AM48

Lati positivi

- La meccanica di gioco sembra essere implementata a tutti gli effetti in modo logico e chiaro.
- Le Enumeration consentono di visualizzare in modo ordinate tutte le variabili/stati di gioco
- Sembrano essere anche state implementate le funzionalità avanzate delle partite multiple e della resilienza delle connessioni

Lati negativi

- **GameManager:**
 - Implementa Serializable quindi dovrebbe implementare funzioni che fanno parsing dei file. Pensiamo faccia troppe cose al di fuori della sua competenza come dealInitialCard(), dealPersonalCard(), askPlayerSelectSideInitialCard(). Secondo noi sono funzioni che dovrebbero essere nel GameModel o in EventHandler.
- **GameModel:**
 - Il field goals secondo noi dovrebbe contenere solo i goal pubblici, e non quelli privati in quanto essi sono specifici per ogni giocatore; quindi, avrebbe più senso implementarli nella classe Player.
 - Non ci è chiara l'utilità del metodo removeGoal(), in quanto supponiamo che nella Lista di Goals della classe siano inseriti solo Goal "attivi" che dovrebbero essere immutabili a runtime.
- **PersonalGoal:**
 - Sottoclasse di Goal, secondo noi potrebbe avere più senso non creare una sottoclasse ma, usando la composizione di oggetti, porre un field Goal all'interno della classe Player.
- **Goal:**
 - Non ci è chiaro il motivo di mettere una String per la condizione della carta Goal.
- **Rule:**
 - Non ci è chiaro a cosa serve rightTurn(Player)
- **Player:**
 - Non ci è chiaro perché placeCard() ritorna un Board dato che getBoard() ha proprio lo scopo di ritornare la Board.
- **Consigli:**
 - Consigliamo di porre GolderCard e ResourceCard come sottoclassi di una Sopraclasse comune che contiene i fields: Value e Kingdom per evitare ripetizione. Non ci è chiaro il perché si è posto la precondition di GoldCard come String. Se l'intenzione era quella di creare una codifica del tipo "FFF" = "Tre farfalle sono necessarie per piazzare questa carta" allora la troviamo una soluzione valida.
- **Angle:**

- Come si identifica un angolo che è disponibile ma non contiene né un oggetto né un Kingdom?

Confronto tra le architetture

Differenze:

- Abbiamo optato a includere oggetti e risorse in una singola enumerazione in quanto diamo per scontato il fatto che l'inserimento delle carte nel JSON è corretto.
- Per lo stesso motivo sopracitato, non abbiamo incluso un'enumeration per i punti attribuiti dalle carte come è stato fatto da AM48 in value, bensì utilizziamo int
- Abbiamo gestito il mazzo del giocatore in modo differente (PlayerDeck), distinguendo i tipi di carte che può avere il giocatore.
- Nella nostra classe Player ci sono due field resourceMana e ElementMana che tengono traccia degli elementi/risorse che il giocatore vede disponibili in ogni momento della partita sul suo tavolo.
- AM48 ha una fase di setup iniziale dove tutti i giocatori piazzano sui loro campi le carte iniziali e solo poi comincia il match. Noi abbiamo un approccio diverso in cui i giocatori, nel loro primo turno, piazzano le loro carte iniziali e una ResourceGoldCard. Nei turni successivi ogni giocatore piazza una carta alla volta. Entrambi gli approcci ci sembrano corretti e non ci sono apparenti i motivi per cui l'implementazione di un metodo sia migliore rispetto all'altra

Modifiche ispirate dall'UML AM48:

- Assegnare un MatchID e MatchStatus per ogni Match.
- Fare la distinzione tra i punti ottenuti durante la partita e i punti delle carte obiettivo personali